



---

# Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

|                    |                           |
|--------------------|---------------------------|
| NIM                | 71251211                  |
| Nama Lengkap       | Jendri Mansamnaf Japen    |
| Minggu ke / Materi | 12 / Tipe Data Dictionary |

PROGRAM STUDI INFORMATIKA  
FAKULTAS TEKNOLOGI INFORMASI  
UNIVERSITAS KRISTEN DUTA WACANA  
YOGYAKARTA  
2026

## BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

### MATERI 12.1

Dictionary mempunyai kemiripan dengan list, tetapi lebih bersifat umum. Dalam list, indeks harus berupa integer sedangkan dalam dictionary indeks bisa dapat berupa apapun. Dictionary memiliki anggota yang terdiri dari pasangan kunci:nilai. Kunci harus bersifat unik, tidak boleh ada dua kunci yang sama dalam satu dictionary. Dictionary dapat diartikan pula sebagai pemetaan antara sekumpulan indeks (yang disebut kunci) dan sekumpulan nilai. Setiap kunci memetakan suatu nilai. Asosiasi kunci dan nilai tersebut disebut dengan pasangan nilai kunci (key-value pair) atau ada yang menyebutnya sebagai item. Sebagai contoh, kita akan mengembangkan kamus yang memetakan dari kata-kata dalam bahasa Inggris ke bahasa Spanyol, jadi bisa dikatakan yang menjadi kunci dan nilai adalah data string. Asumsinya setiap kata dalam bahasa Inggris mempunyai satu arti dalam bahasa Spanyol. Fungsi dict digunakan untuk membuat dictionary baru yang kosong. Karena dict merupakan built-in function dari python, maka penggunaannya perlu dihindari sebagai nama variabel.

#### MATERI 12.1.1 Dictionary sebagai set penghitung (counters)

Sebagai contoh kita diberikan sebuah string dan di haruskan menghitung banyaknya huruf yang muncul, beberapa cara yang bisa digunakan misalnya :

1. Membuat 26 variable untuk tiap huruf pada alfabet, kemudian memasukkannya dalam string untuk masing-masing karakter, menambah perhitungan yang sesuai dan menggunakan kondisional berantai.
2. Membuat list dengan 26 elemen, kemudian melakukan konversi setiap karakter menjadi angka (menggunakan fungsi bawaan), kemudian menggunakan angka sebagai indeks dalam list dan menambah perhitungan yang sesuai.
3. Membuat dictionary dengan karakter sebagai kunci dan perhitungan sebagai nilai yang sesuai, Karakter dapat ditambahkan item kedalam dictionary dan kemudian ditambahkan nilai dari item yang ada.

Dari 3 opsi diatas, kita akan melakukan perhitungan yang sama, akan tetapi dari masing-masing opsi menerapkan proses perhitungan dengan cara yang berbeda-beda.

Implementasi dilakukan dengan menggunakan perhitungan (computation), beberapa perhitungan biasanya lebih baik dari model perhitungan lainnya. Penggunaan komputasi model dictionary lebih praktis, dimana kita tidak perlu mengetahui huruf mana yang akan muncul dalam string. Kita hanya perlu memberikan ruang untuk huruf yang akan muncul. Model penerapannya sebagai berikut:

---

```

1 word = 'brontosaurus'
2 d = dict()
3 for c in word:
4     if c not in d:
5         d[c] = 1
6     else:
7         d[c] = d[c] + 1
8 print(d)

```

---

Model komputasi diatas disebut dengan histogram, yang mana merupakan istilah statistika dari set perhitungan (atau frekuensi).

Perulangan for akan melewati string. Setiap kali terjadi looping, jika karakter c tidak ada dalam dictionary maka akan dibuat item baru dengan kunci c dan nilai awal 1 (asumsinya telah muncul sekali). Jika c sudah ada dalam dictionary secara otomatis akan melakukan penambahan d(c).

Output dari program diatas adalah :

```
{'b': 1, 'r': 2, 'o': 2, 'n': 1, 't': 1, 's': 2, 'a': 1, 'u': 2}
```

Histogram output menunjukkan bahwa huruf "a" dan "b" muncul sekali; "o" muncul dua kali, dan seterusnya.

Dictionary memiliki metode yang disebut get yang mengambil kunci dan nilai default. Jika kunci muncul di dictionary, akan mengembalikan nilai yang sesuai; jika tidak maka akan mengembalikan nilai default. Sebagai contoh:

```

>>> counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
>>> print(counts.get('jan', 0))
100
>>> print(counts.get('tim', 0))
0

```

Penggunaan get dapat dilakukan untuk menulis loop histogram secara lebih ringkas. Metode get secara otomatis menangani case dimana kunci tidak ada dalam dictionary. Dengan menghilangkan statement if, kita dapat meringkas empat baris menjadi satu baris saja.

---

```

1 word = 'brontosaurus'
2 d = dict()
3 for c in word:
4     d[c] = d.get(c,0) + 1
5 print(d)

```

---

dari program diatas adalah :

```
{'b': 1, 'r': 2, 'o': 2, 'n': 1, 't': 1, 's': 2, 'a': 1, 'u': 2}
```

Penggunaan metode get untuk menyederhanakan loop penghitungan ini akhirnya menjadi "idiom" yang sangat umum digunakan dalam Python. Jika dibandingkan loop menggunakan pernyataan

if dan operator in dengan loop menggunakan metode get melakukan hal yang persis sama, tetapi yang satu lebih ringkas.

## MATERI 12.3.2 Dictionary dan File

Salah satu kegunaan umum dictionary adalah untuk menghitung kemunculan kata-kata dalam file dengan beberapa teks tertulis. Mari kita mulai dengan file kata yang sangat sederhana yang diambil dari teks Romeo dan Juliet.

Sebagai contoh pertama, akan digunakan versi teks yang disingkat dan disederhanakan tanpa tanda baca. Selanjutnya akan digunakan teks adegan dengan tanda baca yang disertakan.

*But soft what light through yonder window breaks*

*It is the east and Juliet is the sun*

*Arise fair sun and kill the envious moon*

*Who is already sick and pale with grief*

Kita akan mencoba memnulis program Python untuk membaca baris-baris file, memecah setiap baris menjadi daftar kata, dan kemudian melakukan perulangan melalui setiap kata dalam baris dan menghitung setiap kata menggunakan dictionary.

Asumsikan kita menggunakan dua for untuk perulangan. Perulangan bagian luar untuk membaca baris file dan perulangan pada dalam mekalukan iterasi melalui setiap kata pada baris tertentu. Ini adalah contoh dari pola yang disebut nested loop karena salah satu perulangan adalah perulangan bagian luar dan perulangan lainnya adalah perulangan bagian dalam.

Perulangan dalam melakukan eksekusi pada semua iterasi setiap kali perulangan bagian luar membuat iterasi tunggal. Pada bagian ini, perulangan bagian dalam iterasi "lebih cepat", sedangkan perulangan bagian luar iterasi lebih lambat.

Kombinasi dari dua perulangan bersarang memastikan bahwa ada proses perhitungan setiap kata pada setiap baris file input.

```
1  fname = input('Enter the file name: ')
2  try:
3      fhand = open(fname)
4  except:
5      print('File cannot be opened:', fname)
6      exit()
7
8  counts = dict()
9  for line in fhand:
10     words = line.split()
11     for word in words:
12         if word not in counts:
13             counts[word] = 1
14         else:
15             counts[word] += 1
16
17  print(counts)
```

Pada statement **else** digunakan model penulisan yang lebih singkat untuk menambahkan variable. **counts[word] += 1** sama dengan **counts[word] = counts[word] + 1**. Metode lain dapatdigunakan untuk mengubah nilai suatu variabel dengan jumlah yang diinginkan, misalnya dengan menggunakan -, \*, dan / =.

Ketika kita menjalankan program, akan didapatkan data dump jumlah semua dalam urutan hash yang tidak disortir

### MATERI 12.3.3 Looping dan Dictionary

Dalam statement for, dictionary akan bekerja dengan cara menelusuri kunci yang ada didalamnya. Looping ini akan melakukan pencetakan setiap kunci sesuai dengan hubungan nilainya.

```
1 counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
2 for key in counts:
3     print(key, counts[key])
```

Outputnya akan tampak sebagai berikut:

jan 100

chuck 1

annie 42

Output yang ditampilkan memperlihatkan bahwa kunci tidak berada dalam pola urutan tertentu. Pola ini dapat diimplementasikan dengan berbagai idiom looping yang telah dideskripsikan pada bagian sebelumnya. Sebagai contoh jika ingin menemukan semua entri dalam dictionary dengan nilai diatas 10, maka kode programnya sebagai berikut :

```
1 counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
2 for key in counts:
3     if counts[key] > 10 :
4         print(key, counts[key])
```

Pada looping for yang melalui kunci dictionary, perlu ditambahkan opertor indeks untuk mengambil nilai yang sesuai untuk setiap kunci. Outputnya akan terlihat sebagai berikut :

jan 100

annie 42

Program diatas hanya akan menampilkan data nilai diatas 10.

Untuk melakukan print kunci dalam urutan secara alfabet, langkah pertama yang dilakukan adalah membuat list dari kunci pada dictionary dengan menggunakan method kunci yang tersedia pada object dictionary. Langkah selanjutnya adalah melakukan pengurutan (sort), list dan loop melewati sorted list. Langkah terakhir dengan melihat tiap kunci dan pencetak pasangan nilai key yang sudah diurutkan. Impelentasi dalam kode dapat dilihat dibawah ini:

```
1 counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
2 lst = list(counts.keys())
3 print(lst)
4 lst.sort()
5 for key in lst:
6     print(key, counts[key])
```

Outputnya sebagai berikut:

```
['jan', 'chuck', 'annie']
```

```
annie 42
```

```
chuck 1
```

```
jan 100
```

Pertama-tama kita melihat list dari kunci dengan tidak berurutan yang didapatkan dari method kunci. Kemudian kita melihat pasangan nilai kunci yang disusun secara berurutan menggunakan loop **for**.

### MATERI 12.3.4 Advanced Text Parsing

Pada bagian sebelumnya, kita menggunakan contoh file dimana kalimat pada file tersebut sudah dihilangkan tanda bacanya. Bagian ini kita akan menggunakan file dengan tanda baca lengkap. Dengan menggunakan contoh yang sama yaitu romeo.txt dengan tanda baca lengkap.

```
But, soft! what light through yonder window breaks?  
It is the east, and Juliet is the sun.  
Arise, fair sun, and kill the envious moon,  
Who is already sick and pale with grief,
```

Python sendiri mempunyai function **split** dimana fungsi tersebut akan mencari spasi dan mengubah kata-kata sebagai token yang dipisahkan oleh spasi. Misal pada kata "soft!" dan "soft" merupakan dua kata yang berbeda dan dalam dictionary akan dibuat terpisah untuk tiap kata tersebut.

Hal tersebut juga berlaku pada penggunaan huruf kapital. Misal pada kata "who" dan "Who" dianggap sebagai kata berbeda dan diurutkan secara terpisah.

Model penyelesaian lain dapat juga dengan menggunakan method string lain seperti **lower**, **punctuation** dan **translate**. Jika diperhatikan method string **translate** merupakan method string yang paling stabil (hampir tidak kentara). Berikut ini merupakan dokumentasi dari method **translate**.

```
line.translate(str.maketrans(fromstr, tostr, deletetr))
```

Ganti karakter pada *fromstr* dengan karakter pada posisi yang sama dengan *tostr* dan hapus semua karakter yang ada dalam *deletetr*. Untuk *tostr* dapat berupa string kosong dan untuk parameter pada *deletetr* dapat dihilangkan.

Kita tidak akan menentukan *tostr* tetapi kita akan menggunakan parameter *deletetr* untuk menghapus semua tanda baca. Secara otomatis Python akan memberitahu bagian mana yang dianggap sebagai "tanda baca".

```
1 import string
2
3 fname = input('Enter the file name: ')
4 try:
5     fhand = open(fname)
6 except:
7     print('File cannot be opened:', fname)
8     exit()
9
10 counts = dict()
11 for line in fhand:
12     line = line.rstrip()
13     line = line.translate(line.maketrans('', '', string.punctuation))
14     line = line.lower()
15     words = line.split()
16     for word in words:
17         if word not in counts:
18             counts[word] = 1
19         else:
20             counts[word] += 1
21
22 print(counts)
```

Output dari program diatas adalah :

```
Enter the file name: romeo-full.txt
{'swearst': 1, 'all': 6, 'afearde': 1, 'leave': 2, 'these': 2,
'kinsmen': 2, 'what': 11, 'thinkst': 1, 'love': 24, 'cloak': 1,
a': 24, 'orchard': 2, 'light': 5, 'lovers': 2, 'romeo': 40,
'maiden': 1, 'whiteupturned': 1, 'juliet': 32, 'gentleman': 1,
'it': 22, 'leans': 1, 'canst': 1, 'having': 1, ...}
```

## BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

### SOAL 12.1

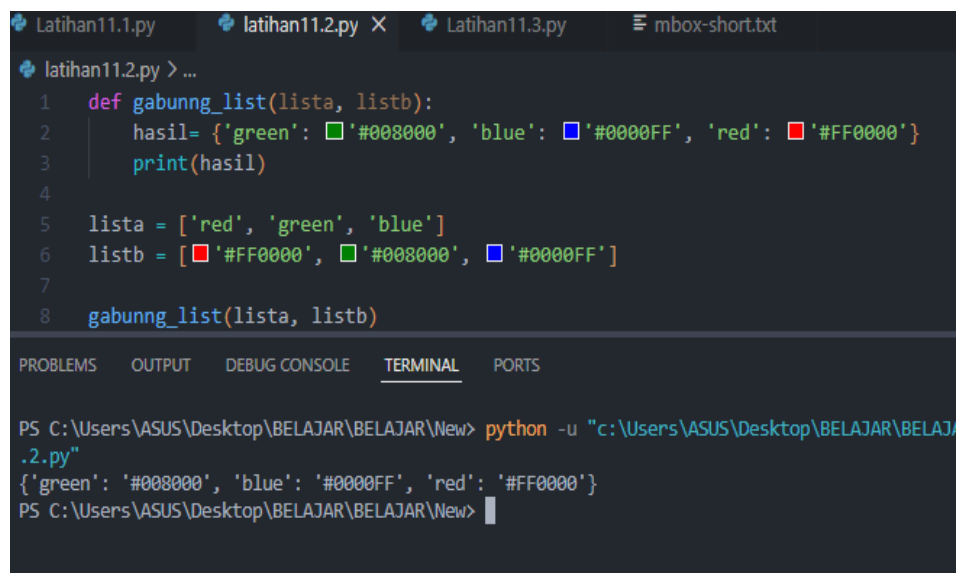


```
Latihan11.1.py X
Latihan11.1.py > ...
1 def tampil_dictionary(data):
2     print("Key value item")
3
4     for key, value in data.items():
5         print(key, " ", value, " ", key, " ")
6 dictionary = {1:10, 2:20, 3:30, 4:40, 5:50, 6:60}
7 tampil_dictionary(dictionary)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\ASUS\Desktop\BELAJAR\BELAJAR\New> python -u "c:\Users\ASUS\Desktop\BELAJAR\BELAJAR\New\Latihan11.1.py"
Key value item
1 10 1
2 20 2
3 30 3
4 40 4
5 50 5
6 60 6
PS C:\Users\ASUS\Desktop\BELAJAR\BELAJAR\New> 
```

Program ini digunakan untuk menampilkan key dan value dari sebuah dictionary. Method items() digunakan untuk mengambil pasangan key dan value secara bersamaan. Hasil akhirnya menampilkan isi dictionary dalam bentuk tabel sederhana.

### SOAL 12.2



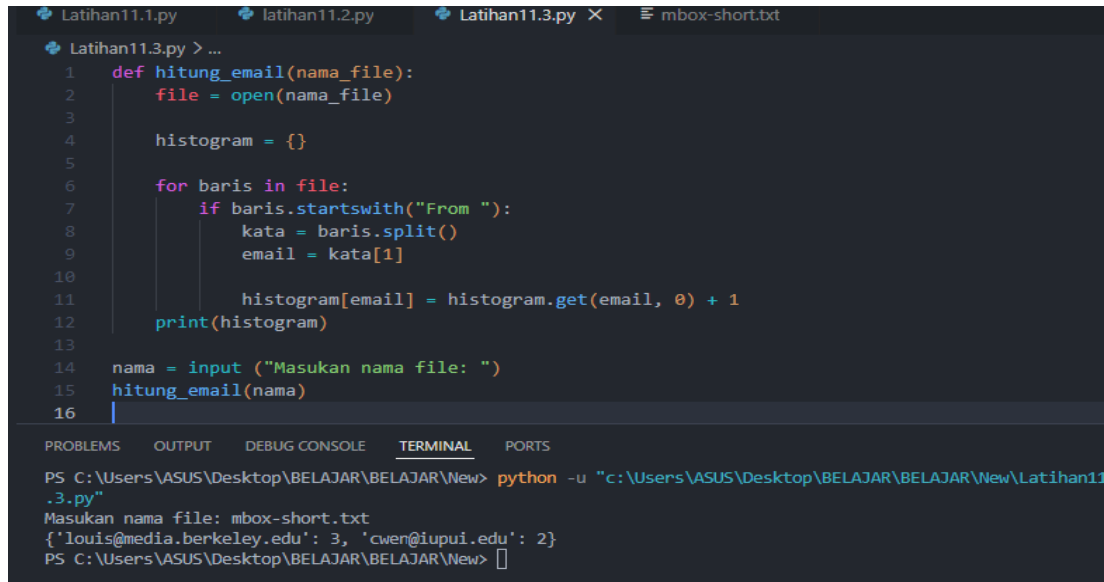
```
Latihan11.1.py latihan11.2.py X Latihan11.3.py mbox-short.txt
latihan11.2.py > ...
1 def gabunng_list(lista, listb):
2     hasil= {'green': '#008000', 'blue': '#0000FF', 'red': '#FF0000'}
3     print(hasil)
4
5     lista = ['red', 'green', 'blue']
6     listb = ['#FF0000', '#008000', '#0000FF']
7
8     gabunng_list(lista, listb)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\ASUS\Desktop\BELAJAR\BELAJAR\New> python -u "c:\Users\ASUS\Desktop\BELAJAR\BELAJAR\New\latihan11.2.py"
{'green': '#008000', 'blue': '#0000FF', 'red': '#FF0000'}
PS C:\Users\ASUS\Desktop\BELAJAR\BELAJAR\New> 
```



Program ini digunakan untuk menggabungkan dua list menjadi dictionary. List pertama menjadi key dan list kedua menjadi value. Hasil akhirnya menampilkan pasangan data warna dan kode warnanya dalam bentuk dictionary.

### SOAL 12.3



```
Latihan11.1.py  latihan11.2.py  Latihan11.3.py X  mbox-short.txt

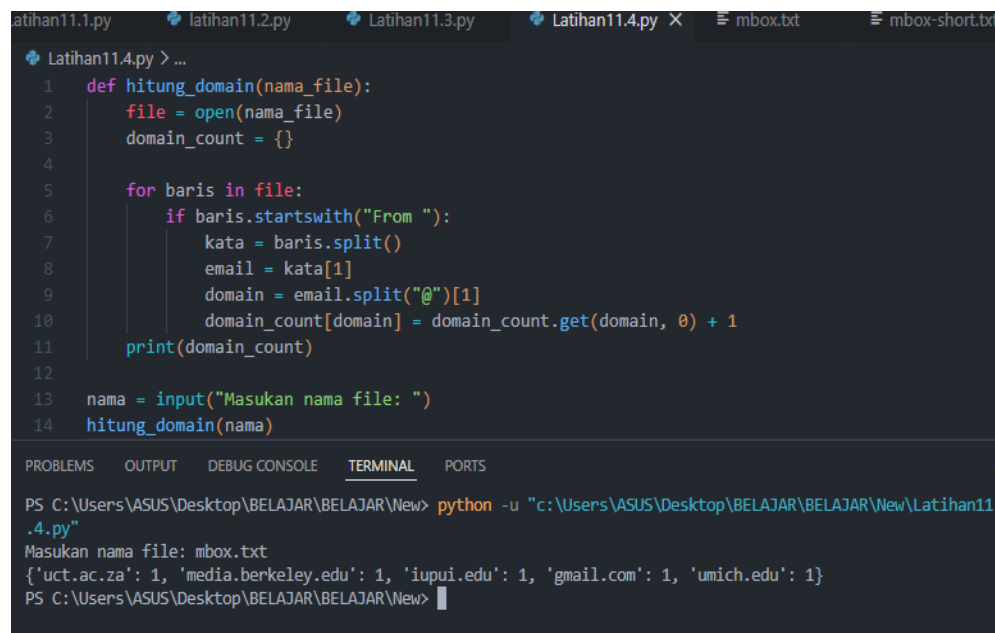
Latihan11.3.py > ...
1  def hitung_email(nama_file):
2      file = open(nama_file)
3
4      histogram = {}
5
6      for baris in file:
7          if baris.startswith("From "):
8              kata = baris.split()
9              email = kata[1]
10
11             histogram[email] = histogram.get(email, 0) + 1
12         print(histogram)
13
14     nama = input("Masukan nama file: ")
15     hitung_email(nama)
16

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\ASUS\Desktop\BELAJAR\BELAJAR\New> python -u "c:\Users\ASUS\Desktop\BELAJAR\BELAJAR\New\Latihan11.3.py"
Masukan nama file: mbox-short.txt
{'louis@media.berkeley.edu': 3, 'cwen@iupui.edu': 2}
PS C:\Users\ASUS\Desktop\BELAJAR\BELAJAR\New> 
```

Program ini menggunakan dictionary untuk membuat histogram email. Setiap baris yang diawali dengan "From " akan diambil alamat emailnya, kemudian dihitung jumlah kemunculannya menggunakan dictionary. Hasil akhirnya menampilkan jumlah pesan yang masuk dari setiap email.

### SOAL 12.4



```
Latihan11.1.py  latihan11.2.py  Latihan11.3.py  Latihan11.4.py X  mbox.txt  mbox-short.txt

Latihan11.4.py > ...
1  def hitung_domain(nama_file):
2      file = open(nama_file)
3      domain_count = {}
4
5      for baris in file:
6          if baris.startswith("From "):
7              kata = baris.split()
8              email = kata[1]
9              domain = email.split("@")[1]
10             domain_count[domain] = domain_count.get(domain, 0) + 1
11         print(domain_count)
12
13     nama = input("Masukan nama file: ")
14     hitung_domain(nama)

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\ASUS\Desktop\BELAJAR\BELAJAR\New> python -u "c:\Users\ASUS\Desktop\BELAJAR\BELAJAR\New\Latihan11.4.py"
Masukan nama file: mbox.txt
{'uct.ac.za': 1, 'media.berkeley.edu': 1, 'iupui.edu': 1, 'gmail.com': 1, 'umich.edu': 1}
PS C:\Users\ASUS\Desktop\BELAJAR\BELAJAR\New> 
```

Program digunakan untuk menghitung jumlah pesan berdasarkan domain email. Program mengambil domain setelah tanda @ pada alamat email, kemudian menghitung

jumlah kemunculannya menggunakan dictionary. Hasil akhirnya menampilkan jumlah pesan dari setiap domain.

Link github: <https://github.com/japenjendri-design/Praktikum-Algoritma-dan-Pemrograman.git>